МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра вычислительной техники

ОТЧЕТ

по лабораторной работе №10
по дисциплине «Организация процессов и программирование в среде Linux»
Тема: Синхронизация процессов с помощью семафоров

Студент гр. 3311 Баймухамедов Р. Р.

Преподаватель Лазарев Э. А.

Санкт-Петербург
2026

ЦЕЛЬ РАБОТЫ

Познакомиться с организацией семафоров в Linux, системными функциями управления ими и применением семафоров для решения задач взаимоисключения и синхронизации процессов.

ЗАДАНИЕ — ЧАСТЬ 2

1. Разработать программы «Писатель» и «Читатель». Писатели с различной частотой дописывают заданную строку в общий файл N раз. Каждый читатель однократно выводит текущее содержимое файла. Обеспечить исключительный доступ писателя, совместное чтение и приоритет ожидающих писателей.
2. Откомпилировать программы «Писатель» и «Читатель», запустить несколько экземпляров на разных терминалах и проверить синхронизацию.

1 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Семафор System V — объект ядра, содержащий неотрицательное целое значение. Набор семафоров доступен разным процессам по идентификатору. Функция semget создаёт набор или получает существующий; semctl устанавливает и считывает значения, а также удаляет набор командой IPC_RMID.

Функция semop выполняет операции, заданные массивом структур sembuf. Поле sem_num задаёт номер семафора, sem_op — операцию, sem_flg — флаги. Положительная операция увеличивает значение, отрицательная уменьшает его при достаточном текущем значении, нулевая ожидает равенства нулю. Все операции одного вызова выполняются атомарно: либо изменяются все нужные значения, либо процесс ожидает возможности выполнить весь набор.

При отсутствии IPC_NOWAIT ожидание является пассивным: процесс блокируется в ядре, не опрашивая семафор в цикле. Флаг SEM_UNDO позволяет ядру компенсировать изменения значения при завершении процесса. В данной реализации его используют для сбалансированных счётчиков активности читателей и писателей.

В задаче «Писатели — Читатели» несколько читателей могут одновременно использовать файл. Запись требует отсутствия и читателей, и другого писателя. При регистрации ожидающего писателя вход новых читателей запрещается; уже работающие читатели завершают чтение. При непрерывном потоке писателей время ожидания читателей может увеличиваться.

2 ПРОГРАММЫ

2.1 Подготовка исходных файлов

Для задачи используются общий файл shared.txt и программы на языке C. Вспомогательные функции вынесены в ipc.h, операции входа и выхода читателей и писателей — в rw.h.

Исходные файлы ipc.h, rw.h, writer.c и reader.c размещены в каталоге ~/lab10. Из этого же каталога выполняются компиляция и запуск программ.

2.2 Общие определения: ipc.h

Заголовок содержит функции проверки аргументов, обработки ошибок, задержки и выполнения semop с повтором при прерывании сигналом. Объединение semun используется для передачи начальных значений в semctl.

```
#ifndef IPC_H
#define IPC_H
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <sys/shm.h>
#include <time.h>
#include <unistd.h>

union semun {
    int val;
    struct semid_ds *buf;
    unsigned short *array;
};

static inline void die(const char *message)
{
    perror(message);
    exit(EXIT_FAILURE);
}
```

```
static inline int number(const char *text)
{
    char *end;
    errno = 0;
    long n = strtol(text, &end, 10);
    if (errno || *text == '\0' || *end || n < 0 || n > INT_MAX) {
        fprintf(stderr, "Invalid number: %s\n", text);
        exit(EXIT_FAILURE);
    }
    return (int)n;
}

static inline void pause_ms(int ms)
{
    struct timespec t = {ms / 1000, (ms % 1000) * 1000000L};
    while (nanosleep(&t, &t) == -1)
        if (errno != EINTR) die("nanosleep");
}

static inline void operations(int id, struct sembuf *ops, size_t n)
{
    while (semop(id, ops, n) == -1)
        if (errno != EINTR) die("semop");
}

#endif
```

2.3 Синхронизация писателей и читателей: rw.h

Для задачи используется набор из трёх семафоров с начальными значениями 0: WAIT_WRITERS — число зарегистрированных ожидающих писателей, ACTIVE_WRITER — наличие работающего писателя, ACTIVE_READERS — число работающих читателей. Ключ набора формируется функцией ftok по общему файлу.

Писатель сначала увеличивает WAIT_WRITERS. Затем одним вызовом semop ожидает отсутствия активных читателей и писателя, уменьшает WAIT_WRITERS и увеличивает ACTIVE_WRITER. Читатель одним вызовом проверяет отсутствие ожидающих и активного писателей и увеличивает ACTIVE_READERS. Атомарность этих действий исключает вход нового читателя между проверкой условий и регистрацией активности.

```
#ifndef RW_H
#define RW_H
#include "ipc.h"

enum { WAIT_WRITERS, ACTIVE_WRITER, ACTIVE_READERS };
```

```
static inline int file_semaphores(const char *path)
{
    key_t key = ftok(path, 'W');
    if (key == (key_t)-1) die("ftok");
    int id = semget(key, 3, 0600);
    if (id == -1) die("semget: run writer --init first");
    return id;
}

static inline void writer_enter(int id)
{
    struct sembuf register_writer = {WAIT_WRITERS, 1, SEM_UNDO};
    operations(id, &register_writer, 1);
    struct sembuf enter[] = {
        {ACTIVE_READERS, 0, 0},
        {ACTIVE_WRITER, 0, 0},
        {WAIT_WRITERS, -1, SEM_UNDO},
        {ACTIVE_WRITER, 1, SEM_UNDO}
    };
    operations(id, enter, 4);
}

static inline void writer_leave(int id)
{
    struct sembuf leave = {ACTIVE_WRITER, -1, SEM_UNDO};
    operations(id, &leave, 1);
}

static inline void reader_enter(int id)
{
    struct sembuf enter[] = {
        {WAIT_WRITERS, 0, 0},
        {ACTIVE_WRITER, 0, 0},
        {ACTIVE_READERS, 1, SEM_UNDO}
    };
    operations(id, enter, 3);
}

static inline void reader_leave(int id)
{
    struct sembuf leave = {ACTIVE_READERS, -1, SEM_UNDO};
    operations(id, &leave, 1);
}
#endif
```

2.4 Программа «Писатель»: writer.c

Режим --init создаёт общий файл при необходимости и инициализирует набор семафоров; его выполняют до запуска рабочих процессов. В рабочем режиме программа N раз дописывает заданную строку, выводя имя программы и PID. DELAY_MS задаёт интервал перед обращением, HOLD_MS —

дополнительную задержку внутри защищённого участка для наблюдения синхронизации. Режим --cleanup удаляет набор после завершения всех участников.

```
#include "rw.h"

int main(int argc, char **argv)
{
    if (argc == 3 && strcmp(argv[1], "--init") == 0) {
        FILE *file = fopen(argv[2], "a");
        if (!file) die("fopen");
        if (fclose(file) == EOF) die("fclose");
        key_t key = ftok(argv[2], 'W');
        if (key == (key_t)-1) die("ftok");
        int id = semget(key, 3, IPC_CREAT | IPC_EXCL | 0600);
        if (id == -1) die("semget --init");
        unsigned short values[] = {0, 0, 0};
        union semun arg = {.array = values};
        if (semctl(id, 0, SETALL, arg) == -1) {
            int saved = errno;
            semctl(id, 0, IPC_RMID);
            errno = saved;
            die("SETALL");
        }
        printf("Initialized: semid=%d\n", id);
        return EXIT_SUCCESS;
    }
    if (argc == 3 && strcmp(argv[1], "--cleanup") == 0) {
        int id = file_semaphores(argv[2]);
        if (semctl(id, 0, IPC_RMID) == -1) die("IPC_RMID");
        puts("Semaphores removed.");
        return EXIT_SUCCESS;
    }
    if (argc < 4 || argc > 6) {
        fprintf(stderr, "Usage: %s FILE TEXT N [DELAY_MS [HOLD_MS]]\n"
                "       %s --init|--cleanup FILE\n", argv[0], argv[0]);
        return EXIT_FAILURE;
    }
    int n = number(argv[3]);
    int delay = argc >= 5 ? number(argv[4]) : 0;
    int hold = argc == 6 ? number(argv[5]) : 0;
    int id = file_semaphores(argv[1]);
    setbuf(stdout, NULL);
    for (int i = 0; i < n; ++i) {
        pause_ms(delay);
        printf("%s pid=%ld: REQUEST %d\n", argv[0], (long)getpid(), i+1);
        writer_enter(id);
        printf("%s pid=%ld: WRITE BEGIN\n", argv[0], (long)getpid());
        FILE *file = fopen(argv[1], "a");
        if (!file) die("fopen");
        if (fprintf(file, "%s\n", argv[2]) < 0) die("fprintf");
        pause_ms(hold);
```

```
        if (fclose(file) == EOF) die("fclose");
        printf("%s pid=%ld: WRITE END: %s\n",
               argv[0], (long)getpid(), argv[2]);
        writer_leave(id);
    }
    return EXIT_SUCCESS;
}
```

2.5 Программа «Читатель»: reader.c

Читатель ожидает разрешения на вход, открывает общий файл и однократно выводит его текущее содержимое. Внутри защищённого участка возможна демонстрационная задержка HOLD_MS. После чтения процесс уменьшает ACTIVE_READERS. Служебные сообщения о начале и конце чтения направляются в стандартный поток ошибок.

```
#include "rw.h"

int main(int argc, char **argv)
{
    if (argc < 2 || argc > 4) {
        fprintf(stderr, "Usage: %s FILE [DELAY_MS [HOLD_MS]]\n", argv[0]);
        return EXIT_FAILURE;
    }
    int delay = argc >= 3 ? number(argv[2]) : 0;
    int hold = argc == 4 ? number(argv[3]) : 0;
    int id = file_semaphores(argv[1]);
    setbuf(stdout, NULL);
    pause_ms(delay);
    fprintf(stderr, "%s pid=%ld: REQUEST\n", argv[0], (long)getpid());
    reader_enter(id);
    fprintf(stderr, "%s pid=%ld: READ BEGIN\n", argv[0], (long)getpid());
    FILE *file = fopen(argv[1], "r");
    if (!file) die("fopen");
    int c;
    while ((c = fgetc(file)) != EOF)
        if (putchar(c) == EOF) die("stdout");
    if (ferror(file)) die("fgetc");
    if (fclose(file) == EOF) die("fclose");
    pause_ms(hold);
    fprintf(stderr, "%s pid=%ld: READ END\n", argv[0], (long)getpid());
    reader_leave(id);
    return EXIT_SUCCESS;
}
```

3 КОМПИЛЯЦИЯ И ЗАПУСК ПРОГРАММ

3.1 Компиляция

Для получения исполняемых файлов использован компилятор gcc. На рисунке 1 показаны компиляция писателя и читателя.

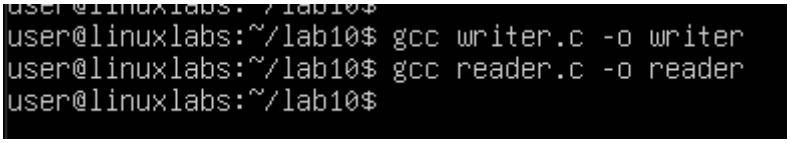


Рисунок 1 - Компиляция писателя и читателя

3.2 Запуск писателей и читателей

Перед началом опыта общий файл shared.txt очищен и создан набор из трёх семафоров. Два экземпляра программы «Писатель» записывают строки Writer A и Writer B с интервалами 500 и 900 мс соответственно. Первый писатель выполняет четыре записи, второй — три. Результаты запусков приведены на рисунках 2 и 3. Два экземпляра программы «Читатель» однократно выводят содержимое общего файла; каждый из них получил семь строк: четыре Writer A и три Writer B (рисунки 4 и 5).

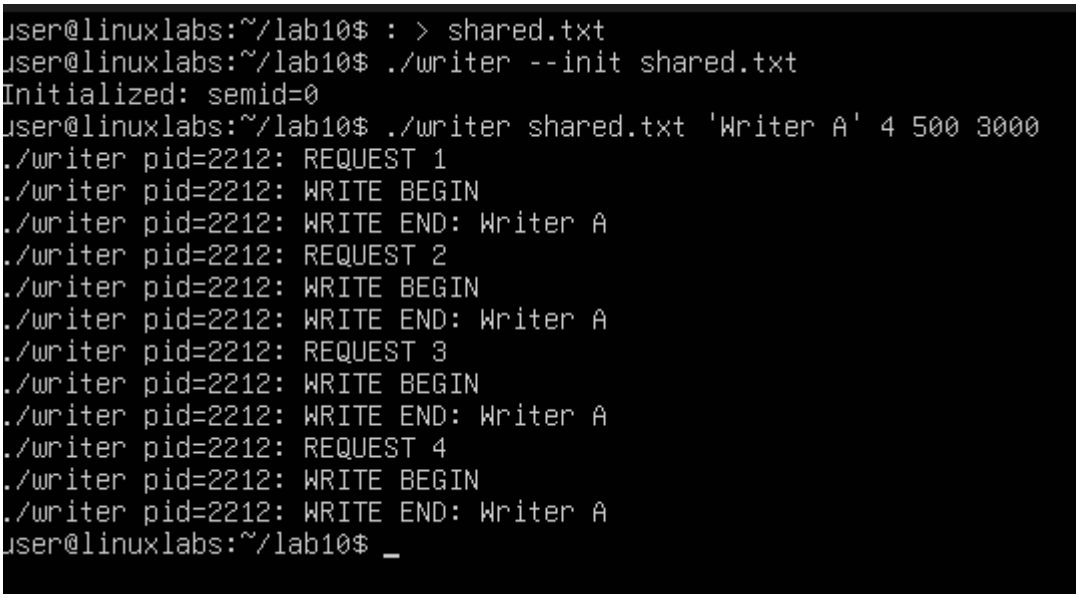


Рисунок 2 - Инициализация семафоров и четыре записи первого писателя

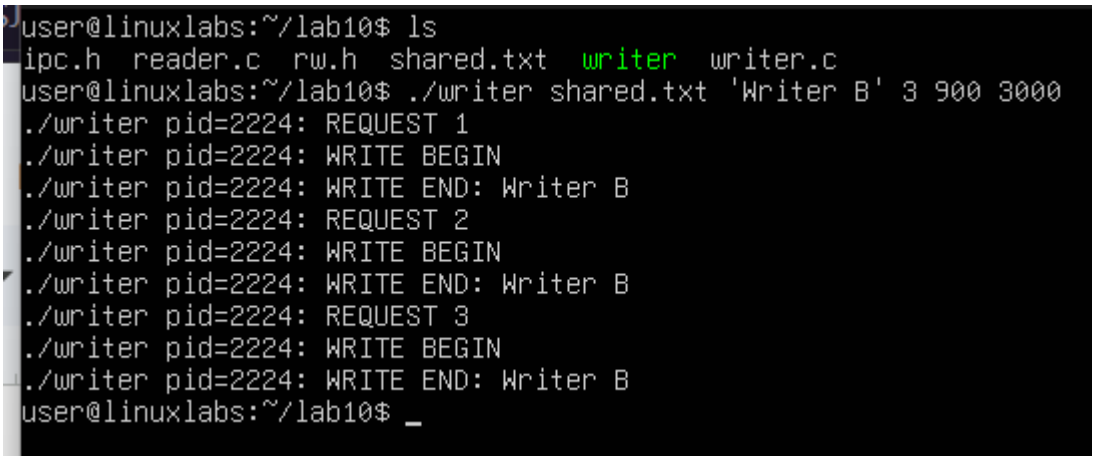


Рисунок 3 - Три записи второго писателя

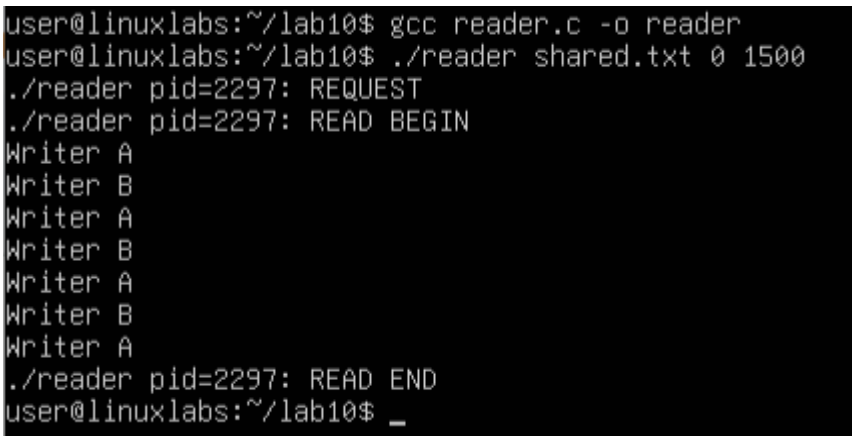


Рисунок 4 - Компиляция и запуск первого читателя

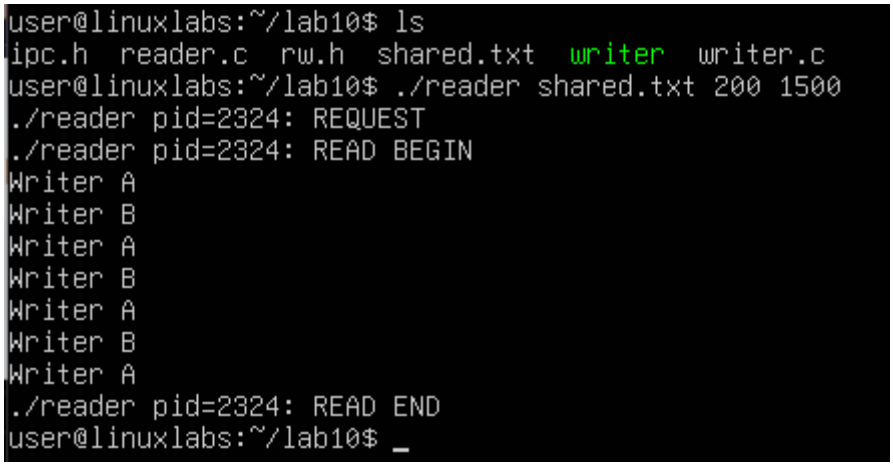


Рисунок 5 - Чтение общего файла вторым читателем

3.3 Проверка приоритета писателей

Для проверки приоритета выполнен сценарий script.sh. Сначала запущены два читателя, удерживающие доступ к файлу в течение 15000 мс. Затем зарегистрирован писатель и запущен третий читатель. Состояния набора семафоров получены командой ipcs -s -i 0 до и после регистрации писателя (рисунок 6).

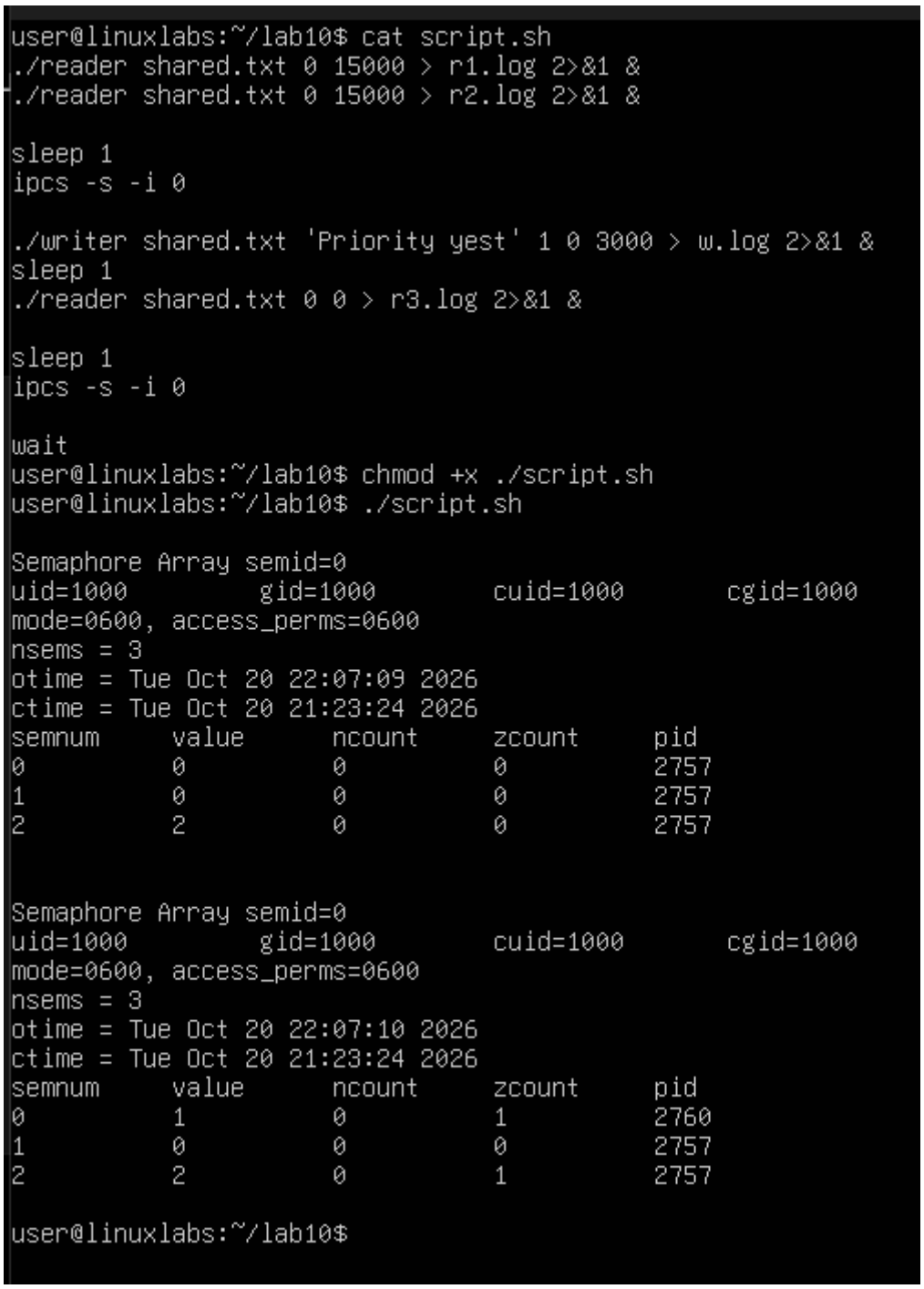


Рисунок 6 - Совместное чтение и блокирование нового читателя при ожидании писателя

В первом выводе ipcs значения WAIT_WRITERS, ACTIVE_WRITER и ACTIVE_READERS равны 0, 0 и 2: два читателя одновременно находятся в защищённом участке. Во втором выводе значения равны 1, 0 и 2: писатель ожидает завершения этих читателей, а новый читатель не получает доступа. Значение zcnt = 1 у семафора 0 означает ожидание новым читателем отсутствия зарегистрированных писателей. Значение zcnt = 1 у семафора 2

соответствует ожиданию писателем завершения активных читателей. Полученное состояние подтверждает приоритет ожидающего писателя перед новым читателем.

4 АНАЛИЗ РЕЗУЛЬТАТОВ

В задаче ACTIVE_WRITER принимает значения 0 или 1, а ACTIVE_READERS может быть больше единицы. Вход писателя допускается только при нулевых значениях обоих счётчиков активности. Если WAIT_WRITERS больше нуля, новые читатели блокируются, что обеспечивает приоритет зарегистрированных ожидающих писателей. Задержка между записями не считается ожиданием доступа и не препятствует чтению.

Читатель получает согласованное содержимое файла на время своего защищённого чтения. Оно не обязано совпадать с итоговым содержимым после всех последующих записей. Закрытие файла писателем выполняется до освобождения доступа, поэтому следующий читатель видит завершённую запись. SEM_UNDO восстанавливает счётчики активности при завершении участника; эта мера не обеспечивает восстановления файла после аварии посередине записи.

ВЫВОД

В ходе лабораторной работы изучены семафоры System V и функции semget, semop и semctl. Разработаны программы «Писатель» и «Читатель». Реализованы взаимоисключение записи, совместное чтение и приоритет ожидающих писателей.